

UNIT-2

Basic Introduction of Stack

Stacks are dynamic data structures that follow the **Last in First out (LIFO)** principle. The last item to be inserted into a stack is the first one to be deleted from it.

For example, you have a stack of trays on a table. The tray at the top of the stack is the first item to be moved if you require a tray from that stack.

Inserting and deleting elements

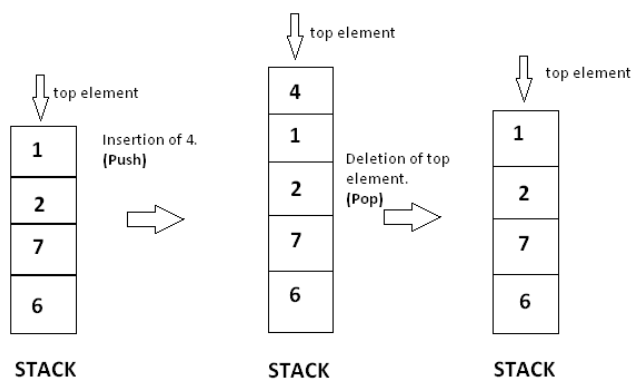
Stacks have restrictions on the insertion and deletion of elements. Elements can be inserted or deleted only from one end of the stack. Element just below the previous top element becomes the new top element of the stack.

For example, in the stack of trays, if you take the tray on the top and do not replace it, then the second tray automatically becomes the top element (tray) of that stack.

Features of stacks

- Dynamic data structures
- Do not have a fixed size
- Do not consume a fixed amount of memory
- Size of stack changes with each push() and pop() operation.
Each push() and pop() operation increases and decreases the size of the stack by 1, respectively.

A stack can be visualized as follows:



Operations

push(x): Insert element x at the top of a stack

```
void push (intstack[],int x ,int n){  
if( top== n-1){//If the top position is the last of position in a stack, this means that the stack is full  
cout<<"Stackisfull.Overflow condition!";  
}
```

```
else{
    top = top + 1; //Incrementing top position
    stack[ top ] = x ; //Inserting element on incremented position
}
}
```

pop(): Removes an element from the top of a stack

```
void pop (intstack[],int n )
{
    if(isEmpty())
    {
        cout<<"Stack is empty.Underflow condition!"<<endl;
    }
    else
    {
        top = top -1; //Decrementing top's position will detach last element from stack
    }
}
```

topElement(): Access the top element of a stack

```
int topElement()
{
    return stack[ top ];
}
```

isEmpty() : Check whether a stack is empty

```
bool isEmpty()
{
    if( top == -1) //Stack is empty
        return true;
    else
        return false;
}
```

size (): Determines the current size of a stack

```
int size ()
{
    return top + 1;
}
```

Introduction To Queues

A queue is an ordered collection of items where the addition of new items happens at one end, called the “rear,” and the removal of existing items occurs at the other end, commonly called the “front.” As an element enters the queue it starts at the rear and makes its way toward the front, waiting until that time when it is the next element to be removed.

The most recently added item in the queue must wait at the end of the collection. The item that has been in the collection the longest is at the front. This ordering principle is sometimes called **FIFO, first-in first-out**. It is also known as “first-come first-served.”

The simplest example of a queue is the typical line that we all participate in from time to time. We wait in a line for a movie, we wait in the check-out line at a grocery store, and we wait in the cafeteria line (so that we can pop the tray stack). Well-behaved lines, or queues, are very restrictive in that they have only one way in and only one way out. There is no jumping in the middle and no leaving before you have waited the necessary amount of time to get to the front.



A queue of Python data objects

Queue Abstract Data Type

A queue is structured as an ordered collection of items which are added at one end, called the “rear,” and removed from the other end, called the “front.” The queue operations are:

- Queue() creates a new queue that is empty. It needs no parameters and returns an empty queue.
- enqueue(item) adds a new item to the rear of the queue. It needs the item and returns nothing.
- dequeue() removes the front item from the queue. It needs no parameters and returns the item. The queue is modified.
- is_empty() tests to see whether the queue is empty. It needs no parameters and returns a boolean value.
- size() returns the number of items in the queue. It needs no parameters and returns an integer.

Types of Queue

There are four different types of queue that are listed as follows -

- o Simple Queue or Linear Queue
- o Circular Queue
- o Priority Queue
- o Double Ended Queue (or Deque)

Simple Queue or Linear Queue

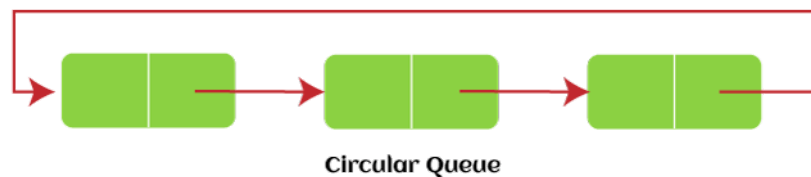
In Linear Queue, an insertion takes place from one end while the deletion occurs from another end. The end at which the insertion takes place is known as the rear end, and the end at which the deletion takes place is known as front end. It strictly follows the FIFO rule.



The major drawback of using a linear Queue is that insertion is done only from the rear end. If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue. In this case, the linear Queue shows the overflow condition as the rear is pointing to the last element of the Queue.

Circular Queue

In Circular Queue, all the nodes are represented as circular. It is similar to the linear Queue except that the last element of the queue is connected to the first element. It is also known as Ring Buffer, as all the ends are connected to another end. The representation of circular queue is shown in the below image -



The drawback that occurs in a linear queue is overcome by using the circular queue. If the empty space is available in a circular queue, the new element can be added in an empty space by simply incrementing the value of rear. The main advantage of using the circular queue is better memory utilization.

Priority Queue

It is a special type of queue in which the elements are arranged based on the priority. It is a special type of queue data structure in which every element has a priority associated with it. Suppose some elements occur with the same priority, they will be arranged according to the FIFO principle. The representation of priority queue is shown in the below image -



Insertion in priority queue takes place based on the arrival, while deletion in the priority queue occurs based on the priority. Priority queue is mainly used to implement the CPU scheduling algorithms.

There are two types of priority queue that are discussed as follows -

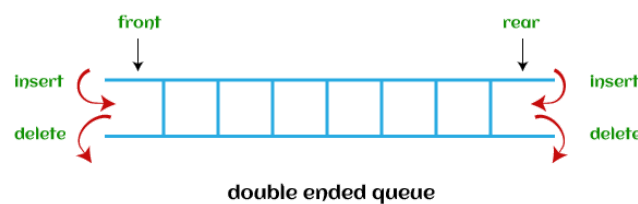
- o **Ascending priority queue** - In ascending priority queue, elements can be inserted in arbitrary order, but only smallest can be deleted first. Suppose an array with elements 7, 5, and 3 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 3, 5, 7.
- o **Descending priority queue** - In descending priority queue, elements can be inserted in arbitrary order, but only the largest element can be deleted first. Suppose an array with elements 7, 3, and 5 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 7, 5, 3.

Deque (or, Double Ended Queue)

In Deque or Double Ended Queue, insertion and deletion can be done from both ends of the queue either from the front or rear. It means that we can insert and delete elements from both front and rear ends of the queue. Deque can be used as a palindrome checker means that if we read the string from both ends, then the string would be the same.

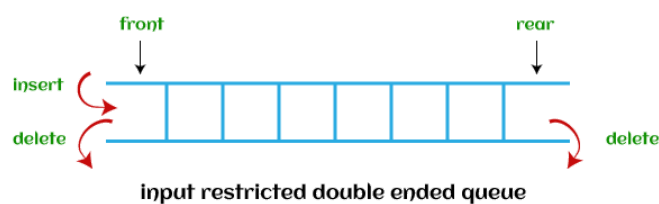
Deque can be used both as stack and queue as it allows the insertion and deletion operations on both ends. Deque can be considered as stack because stack follows the LIFO (Last In First Out) principle in which insertion and deletion both can be performed only from one end. And in deque, it is possible to perform both insertion and deletion from one end, and Deque does not follow the FIFO principle.

The representation of the deque is shown in the below image -

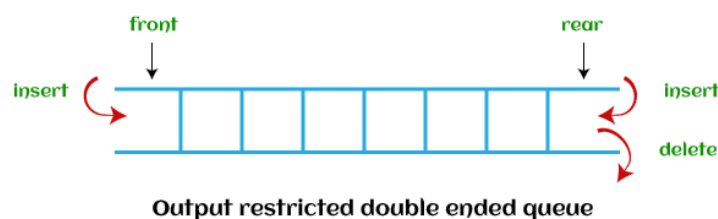


There are two types of deque that are discussed as follows -

- o **Input restricted deque** - As the name implies, in input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



- o **Output restricted deque** - As the name implies, in output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



Operations performed on queue

The fundamental operations that can be performed on queue are listed as follows -

- o **Enqueue:** The Enqueue operation is used to insert the element at the rear end of the queue. It returns void.
- o **Dequeue:** It performs the deletion from the front-end of the queue. It also returns the element which has been removed from the front-end. It returns an integer value.
- o **Peek:** This is the third operation that returns the element, which is pointed by the front pointer in the queue but does not delete it.
- o **Queue overflow (isfull):** It shows the overflow condition when the queue is completely full.
- o **Queue underflow (isempty):** It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

Ways to implement the queue

There are two ways of implementing the Queue:

- o **Implementation using array:**
- o **Implementation using Linked list:**

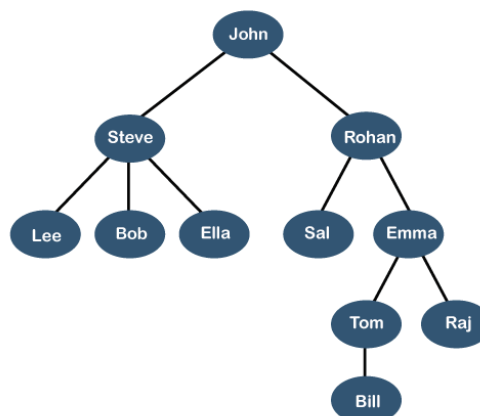
Tree Data Structure

We read the linear data structures like an array, linked list, stack and queue in which all the elements are arranged in a sequential manner. The different data structures are used for different kinds of data.

Some factors are considered for choosing the data structure:

- o **What type of data needs to be stored?:** It might be a possibility that a certain data structure can be the best fit for some kind of data.
- o **Cost of operations:** If we want to minimize the cost for the operations for the most frequently performed operations. For example, we have a simple list on which we have to perform the search operation; then, we can create an array in which elements are stored in sorted order to perform the **binary search**. The binary search works very fast for the simple list as it divides the search space into half.
- o **Memory usage:** Sometimes, we want a data structure that utilizes less memory.

A tree is also one of the data structures that represent hierarchical data. Suppose we want to show the employees and their positions in the hierarchical form then it can be represented as shown below:

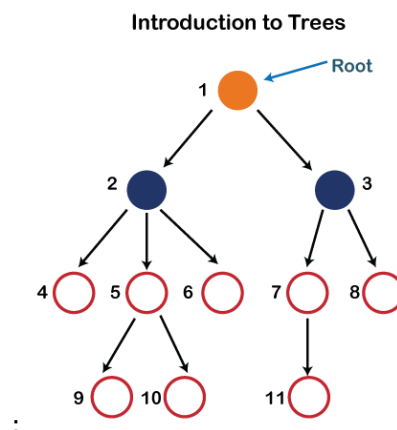


The above tree shows the **organization hierarchy** of some company. In the above structure, **John** is the **CEO** of the company, and John has two direct reports named as **Steve** and **Rohan**. Steve has three direct reports named **Lee**, **Bob**, **Ella** where **Steve** is a manager. Bob has two direct reports named **Sal** and **Emma**. **Emma** has two direct reports named **Tom** and **Raj**. Tom has one direct report named **Bill**. This particular logical structure is known as a **Tree**. Its structure is similar to the real tree, so it is named a **Tree**. In this structure, the **root** is at the top, and its branches are moving in a downward direction. Therefore, we can say that the Tree data structure is an efficient way of storing the data in a hierarchical way.

Let's understand some key points of the Tree data structure.

- o A tree data structure is defined as a collection of objects or entities known as nodes that are linked together to represent or simulate hierarchy.
- o A tree data structure is a non-linear data structure because it does not store in a sequential manner. It is a hierarchical structure as elements in a Tree are arranged in multiple levels.
- o In the Tree data structure, the topmost node is known as a root node. Each node contains some data, and data can be of any type. In the above tree structure, the node contains the name of the employee, so the type of data would be a string.
- o Each node contains some data and the link or reference of other nodes that can be called children.

Some basic terms used in Tree data structure.



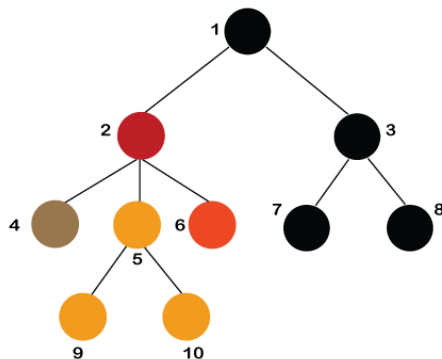
In the above structure, each node is labeled with some number. Each arrow shown in the above figure is known as a **link** between the two nodes.

- o **Root:** The root node is the topmost node in the tree hierarchy. In other words, the root node is the one that doesn't have any parent. In the above structure, node numbered 1 is **the root node of the tree**. If a node is directly linked to some other node, it would be called a parent-child relationship.
- o **Child node:** If the node is a descendant of any node, then the node is known as a child node.

- o **Parent:** If the node contains any sub-node, then that node is said to be the parent of that sub-node.
- o **Sibling:** The nodes that have the same parent are known as siblings.
- o **Leaf Node:-** The node of the tree, which doesn't have any child node, is called a leaf node. A leaf node is the bottom-most node of the tree. There can be any number of leaf nodes present in a general tree. Leaf nodes can also be called external nodes.
- o **Internal nodes:** A node has atleast one child node known as an *internal*
- o **Ancestor node:-** An ancestor of a node is any predecessor node on a path from the root to that node. The root node doesn't have any ancestors. In the tree shown in the above image, nodes 1, 2, and 5 are the ancestors of node 10.
- o **Descendant:** The immediate successor of the given node is known as a descendant of a node. In the above figure, 10 is the descendant of node 5.

Properties of Tree data structure

- o **Recursive data structure:** The tree is also known as a *recursive data structure*. A tree can be defined as recursively because the distinguished node in a tree data structure is known as a **root node**. The root node of the tree contains a link to all the roots of its subtrees. The left subtree is shown in the yellow color in the below figure, and the right subtree is shown in the red color. The left subtree can be further split into subtrees shown in three different colors. Recursion means reducing something in a self-similar manner. So, this recursive property of the tree data structure is implemented in various applications.

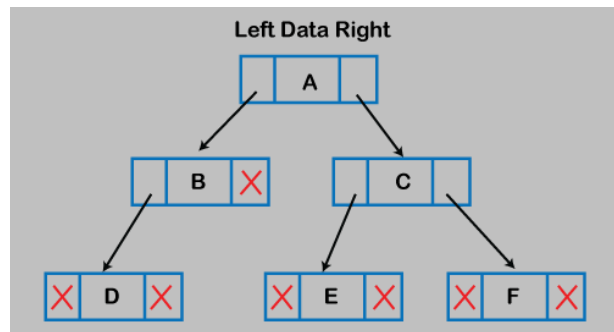


- o **Number of edges:** If there are n nodes, then there would be $n-1$ edges. Each arrow in the structure represents the link or path. Each node, except the root node, will have atleast one incoming link known as an edge. There would be one link for the parent-child relationship.
- o **Depth of node x:** The depth of node x can be defined as the length of the path from the root to the node x . One edge contributes one-unit length in the path. So, the depth of node x can also be defined as the number of edges between the root node and the node x . The root node has 0 depth.
- o **Height of node x:** The height of node x can be defined as the longest path from the node x to the leaf node.

Based on the properties of the Tree data structure, trees are classified into various categories.

Implementation of Tree

The tree data structure can be created by creating the nodes dynamically with the help of the pointers. The tree in the memory can be represented as shown below:



The above figure shows the representation of the tree data structure in the memory. In the above structure, the node contains three fields. The second field stores the data; the first field stores the address of the left child, and the third field stores the address of the right child.

In programming, the structure of a node can be defined as:

```
struct node
{
    int data;
    struct node *left;
    struct node *right;
}
```

The above structure can only be defined for the binary trees because the binary tree can have utmost two children, and generic trees can have more than two children. The structure of the node for generic trees would be different as compared to the binary tree.

Applications of Trees

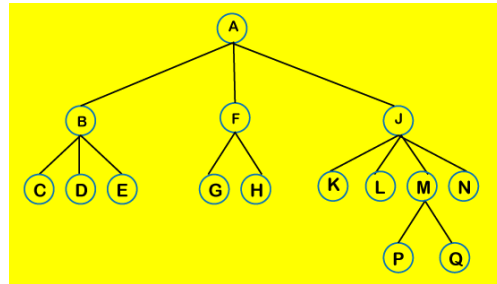
The following are the applications of trees:

- o **Storing naturally hierarchical data:** Trees are used to store the data in the hierarchical structure. For example, the file system. The file system stored on the disc drive, the file and folder are in the form of the naturally hierarchical data and stored in the form of trees.
- o **Organize data:** It is used to organize data for efficient insertion, deletion and searching. For example, a binary tree has a $\log N$ time for searching an element.
- o **Trie:** It is a special kind of tree that is used to store the dictionary. It is a fast and efficient way for dynamic spell checking.
- o **Heap:** It is also a tree data structure implemented using arrays. It is used to implement priority queues.
- o **B-Tree and B+Tree:** B-Tree and B+Tree are the tree data structures used to implement indexing in databases.
- o **Routing table:** The tree data structure is also used to store the data in routing tables in the routers.

Types of Tree Data Structure

The following are the types of a tree data structure:

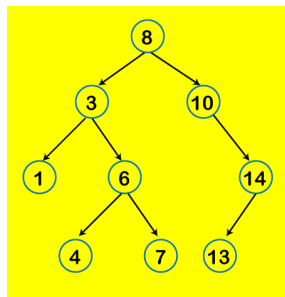
- o **General tree:** The general tree is one of the types of tree data structure. In the general tree, a node can have either 0 or maximum n number of nodes. There is no restriction imposed on the degree of the node (the number of nodes that a node can contain). The topmost node in a general tree is known as a root node. The children of the parent node are known as **subtrees**.



There can be n number of subtrees in a general tree. In the general tree, the subtrees are unordered as the nodes in the subtree cannot be ordered.

Every non-empty tree has a downward edge, and these edges are connected to the nodes known as **child nodes**. The root node is labeled with level 0. The nodes that have the same parent are known as **siblings**.

- o **Binary tree:** Here, binary name itself suggests two numbers, i.e., 0 and 1. In a binary tree, each node in a tree can have utmost two child nodes. Here, utmost means whether the node has 0 nodes, 1 node or 2 nodes.



- o **Binary Search tree:** Binary search tree is a non-linear data structure in which one node is connected to n number of nodes. It is a node-based data structure. A node can be represented in a binary search tree with three fields, i.e., data part, left-child, and right-child. A node can be connected to the utmost two child nodes in a binary search tree, so the node contains two pointers (left child and right child pointer). Every node in the left subtree must contain a value less than the value of the root node, and the value of each node in the right subtree must be bigger than the value of the root node.

A node can be created with the help of a user-defined data type known as **struct**, as shown below:

```

struct node
{
    int data;
    struct node *left;
    struct node *right;
}
  
```

The above is the node structure with three fields: data field, the second field is the left pointer of the node type, and the third field is the right pointer of the node type.

- o **AVL tree**

It is one of the types of the binary tree, or we can say that it is a variant of the binary search tree. AVL tree satisfies the property of the **binary tree** as well as of the **binary search tree**. It is a self-balancing binary search tree that was invented by **Adelson Velsky** and **Lindas**. Here, self-balancing means that balancing the heights of left subtree and right subtree. This balancing is measured in terms of the **balancing factor**.

We can consider a tree as an AVL tree if the tree obeys the binary search tree as well as a balancing factor. The balancing factor can be defined as the **difference between the height of the left subtree and the height of the right subtree**. The balancing factor's value must be either 0, -1, or 1; therefore, each node in the AVL tree should have the value of the balancing factor either as 0, -1, or 1.

- o **Red-Black Tree**

The red-Black tree is the binary search tree. The prerequisite of the Red-Black tree is that we should know about the binary search tree. In a binary search tree, the value of the left-subtree should be less than the value of that node, and the value of the right-subtree should be greater than the value of that node. As we know that the time complexity of binary search in the average case is $\log_2 n$, the best case is $O(1)$, and the worst case is $O(n)$.

When any operation is performed on the tree, we want our tree to be balanced so that all the operations like searching, insertion, deletion, etc., take less time, and all these operations will have the time complexity of **$\log_2 n$** .

The red-black tree is a self-balancing binary search tree. AVL tree is also a height balancing binary search tree then **why do we require a Red-Black tree**. In the AVL tree, we do not know how many rotations would be required to balance the tree, but in the Red-black tree, a maximum of 2 rotations are required to balance the tree. It contains one extra bit that represents either the red or black color of a node to ensure the balancing of the tree.

- o **Splay tree**

The splay tree data structure is also binary search tree in which recently accessed element is placed at the root position of tree by performing some rotation operations. Here, **splaying** means the recently accessed node. It is a **self-balancing** binary search tree having no explicit balance condition like **AVL** tree.

It might be a possibility that height of the splay tree is not balanced, i.e., height of both left and right subtrees may differ, but the operations in splay tree takes order of **$\log N$** time where **n** is the number of nodes.

Splay tree is a balanced tree but it cannot be considered as a height balanced tree because after each operation, rotation is performed which leads to a balanced tree.

- o **B-tree**

B-tree is a balanced **m-way** tree where **m** defines the order of the tree. Till now, we read that the node contains only one key but b-tree can have more than one key, and more than 2 children. It always maintains the sorted data. In binary tree, it is possible that leaf nodes can be at different levels, but in b-tree, all the leaf nodes must be at the same level.

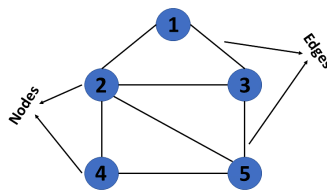
If order is m then node has the following properties:

- o Each node in a b-tree can have maximum m children
- o For minimum children, a leaf node has 0 children, root node has minimum 2 children and internal node has minimum ceiling of $m/2$ children. For example, the value of m is 5 which means that a node can have 5 children and internal nodes can contain maximum 3 children.
- o Each node has maximum $(m-1)$ keys.

The root node must contain minimum 1 key and all other nodes must contain atleast **ceiling of $m/2$ minus 1** keys.

Introduction To Graphs**What Are Graphs in Data Structure?**

A graph is a non-linear kind of data structure made up of nodes or vertices and edges. The edges connect any two nodes in the graph, and the nodes are also known as vertices.



This graph has a set of vertices $V = \{1, 2, 3, 4, 5\}$ and a set of edges $E = \{(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (4, 5)\}$.

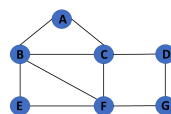
Now that you've learned about the definition of graphs in data structures, you will learn about their various types.

Types of Graphs in Data Structures

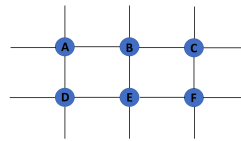
There are different types of graphs in data structures, each of which is detailed below.

1. Finite Graph

The graph $G = (V, E)$ is called a finite graph if the number of vertices and edges in the graph is limited in number

**2. Infinite Graph**

The graph $G=(V, E)$ is called a finite graph if the number of vertices and edges in the graph is interminable.



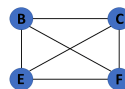
3. Trivial Graph

A graph $G= (V, E)$ is trivial if it contains only a single vertex and no edges.



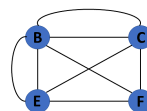
4. Simple Graph

If each pair of nodes or vertices in a graph $G=(V, E)$ has only one edge, it is a simple graph. As a result, there is just one edge linking two vertices, depicting one-to-one interactions between two elements.



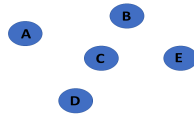
5. Multi Graph

If there are numerous edges between a pair of vertices in a graph $G= (V, E)$, the graph is referred to as a multigraph. There are no self-loops in a Multigraph.



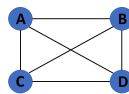
6. Null Graph

It's a reworked version of a trivial graph. If several vertices but no edges connect them, a graph $G= (V, E)$ is a null graph.



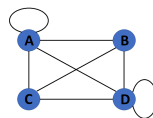
7. Complete Graph

If a graph $G = (V, E)$ is also a simple graph, it is complete. Using the edges, with n number of vertices must be connected. It's also known as a full graph because each vertex's degree must be $n-1$.



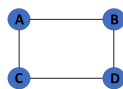
8. Pseudo Graph

If a graph $G = (V, E)$ contains a self-loop besides other edges, it is a pseudograph.



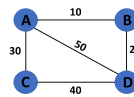
9. Regular Graph

If a graph $G = (V, E)$ is a simple graph with the same degree at each vertex, it is a regular graph. As a result, every whole graph is a regular graph.



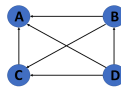
10. Weighted Graph

A graph $G = (V, E)$ is called a labeled or weighted graph because each edge has a value or weight representing the cost of traversing that edge.



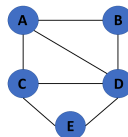
11. Directed Graph

A directed graph also referred to as a digraph, is a set of nodes connected by edges, each with a direction.



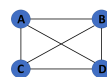
12. Undirected Graph

An undirected graph comprises a set of nodes and links connecting them. The order of the two connected vertices is irrelevant and has no direction. You can form an undirected graph with a finite number of vertices and edges.



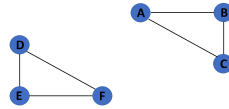
13. Connected Graph

If there is a path between one vertex of a graph data structure and any other vertex, the graph is connected.



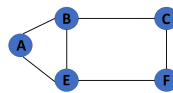
14. Disconnected Graph

When there is no edge linking the vertices, you refer to the null graph as a disconnected graph.



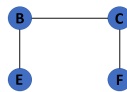
15. Cyclic Graph

If a graph contains at least one graph cycle, it is considered to be cyclic.



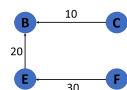
16. Acyclic Graph

When there are no cycles in a graph, it is called an acyclic graph.



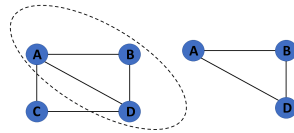
17. Directed Acyclic Graph

It's also known as a directed acyclic graph (DAG), and it's a graph with directed edges but no cycle. It represents the edges using an ordered pair of vertices since it directs the vertices and stores some data.



18. Subgraph

The vertices and edges of a graph that are subsets of another graph are known as a subgraph.



After you learn about the many types of graphs in graphs in data structures, you will move on to graph terminologies.

Terminologies of Graphs in Data Structures

Following are the basic terminologies of graphs in data structures:

- An edge is one of the two primary units used to form graphs. Each edge has two ends, which are vertices to which it is attached.
- If two vertices are endpoints of the same edge, they are adjacent.
- A vertex's outgoing edges are directed edges that point to the origin.
- A vertex's incoming edges are directed edges that point to the vertex's destination.
- The total number of edges occurring to a vertex in a graph is its degree.
- The out-degree of a vertex in a directed graph is the total number of outgoing edges, whereas the in-degree is the total number of incoming edges.
- A vertex with an in-degree of zero is referred to as a source vertex, while one with an out-degree of zero is known as sink vertex.
- An isolated vertex is a zero-degree vertex that is not an edge's endpoint.
- A path is a set of alternating vertices and edges, with each vertex connected by an edge.
- The path that starts and finishes at the same vertex is known as a cycle.
- A path with unique vertices is called a simple path.
- For each pair of vertices x, y , a graph is strongly connected if it contains a directed path from x to y and a directed path from y to x .
- A directed graph is weakly connected if all of its directed edges are replaced with undirected edges, resulting in a connected graph. A weakly linked graph's vertices have at least one out-degree or in-degree.
- A tree is a connected forest. The primary form of the tree is called a rooted tree, which is a free tree.
- A spanning subgraph that is also a tree is known as a spanning tree.
- A connected component is the unconnected graph's most connected subgraph.
- A bridge, which is an edge of removal, would sever the graph.
- Forest is a graph without a cycle.

Following that, you will look at the graph representation in this data structures tutorial.

Operations on Graphs in Data Structures

The operations you perform on the graphs in data structures are listed below:

- Creating graphs
- Insert vertex
- Delete vertex
- Insert edge
- Delete edge

Graph Traversal Algorithm

The process of visiting or updating each vertex in a graph is known as graph traversal. The sequence in which they visit the vertices is used to classify such traversals. Graph traversal is a subset of tree traversal.

There are two techniques to implement a graph traversal algorithm:

- Breadth-first search
- Depth-first search

Breadth-First Search or BFS

BFS is a search technique for finding a node in a graph data structure that meets a set of criteria.

- It begins at the root of the graph and investigates all nodes at the current depth level before moving on to nodes at the next depth level.
- To maintain track of the child nodes that have been encountered but not yet inspected, more memory, generally you require a queue.

Algorithm of breadth-first search

Step 1: Consider the graph you want to navigate.

Step 2: Select any vertex in your graph, say v1, from which you want to traverse the graph.

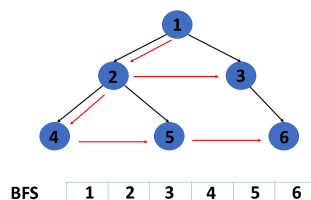
Step 3: Examine any two data structures for traversing the graph.

- Visited array (size of the graph)
- Queue data structure

Step 4: Starting from the vertex, you will add to the visited array, and afterward, you will v1's adjacent vertices to the queue data structure.

Step 5: Now, using the FIFO concept, you must remove the element from the queue, put it into the visited array, and then return to the queue to add the adjacent vertices of the removed element.

Step 6: Repeat step 5 until the queue is not empty and no vertex is left to be visited.



Depth-First Search or DFS

DFS is a search technique for finding a node in a graph data structure that meets a set of criteria.

- The depth-first search (DFS) algorithm traverses or explores data structures such as trees and graphs. The DFS algorithm begins at the root node and examines each branch as far as feasible before backtracking.
- To maintain track of the child nodes that have been encountered but not yet inspected, more memory, generally a stack, is required.

Algorithm of depth-first search

Step 1: Consider the graph you want to navigate.

Step 2: Select any vertex in our graph, say v1, from which you want to begin traversing the graph.

Step 3: Examine any two data structures for traversing the graph.

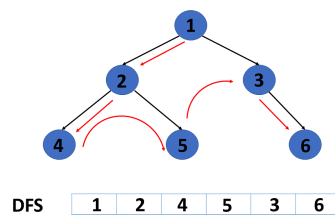
- Visited array (size of the graph)
- Stack data structure

Step 4: Insert v1 into the array's first block and push all the adjacent nodes or vertices of vertex v1 into the stack.

Step 5: Now, using the FIFO principle, pop the topmost element and put it into the visited array, pushing all of the popped element's nearby nodes into it.

Step 6: If the topmost element of the stack is already present in the array, discard it instead of inserting it into the visited array.

Step 7: Repeat step 6 until the stack data structure isn't empty.



Application of Graphs in Data Structures

Following are some applications of graphs in data structures:

- Graphs are used in computer science to depict the flow of computation.
- Users on Facebook are referred to as vertices, and if they are friends, there is an edge connecting them. The Friend Suggestion system on Facebook is based on graph theory.
- You come across the Resource Allocation Graph in the Operating System, where each process and resource are regarded vertically. Edges are drawn from resources to assigned functions or from the requesting process to the desired resource. A stalemate will develop if this results in the establishment of a cycle.
- Web pages are referred to as vertices on the World Wide Web. Suppose there is a link from page A to page B that can represent an edge. This application is an illustration of a directed graph.
- Graph transformation systems manipulate graphs in memory using rules. Graph databases store and query graph-structured data in a transaction-safe, permanent manner.

GENERAL SEARCH ALGORITHM

There are several general search algorithms used in computer science and artificial intelligence to find a solution or locate a specific element within a dataset.

Searching for Solution in AI

Searching for a solution in AI involves employing search algorithms to navigate through a problem space and find a sequence of actions or a configuration that satisfies a goal. The

problem space represents all possible states and transitions between states in a given scenario. Here are the key steps involved in the searching process in AI:

Problem Formulation: Define the problem by specifying the initial state, possible actions, transition model (how actions affect the state), goal state, and the cost function (if applicable).

State Space Representation: Create a representation of the state space, encompassing all possible states the system can be in. The state space is essential for guiding the search process.

Search Algorithm Selection: Choose an appropriate search algorithm based on the characteristics of the problem. Common algorithms include Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search, and A* Search Algorithm.

Heuristic Function (if using informed search): If using informed search algorithms like A*, define a heuristic function that estimates the cost from the current state to the goal. The heuristic guides the search towards potentially promising paths.

Search Strategy: Determine the strategy for exploring the state space. This includes decisions on how to prioritize or order the expansion of nodes in the search tree.

Search Execution: Apply the chosen search algorithm to explore the state space. The search algorithm systematically generates and explores states until a goal state is reached.

Solution Extraction: Once a solution is found, extract the sequence of actions or the configuration that leads from the initial state to the goal state.

Solution Evaluation: Evaluate the quality of the solution based on predefined criteria, such as optimality, efficiency, or domain-specific metrics.

Iterative Refinement (if needed): Depending on the problem complexity or search performance, iteratively refine the search process or algorithm selection.

Learning and Adaptation (if applicable): Some AI systems incorporate learning mechanisms to adapt and improve their search strategies based on experience and feedback.

Searching for solutions in AI is a fundamental aspect of problem-solving and is applicable in various domains, including robotics, planning, game playing, natural language processing, and optimization problems. The effectiveness of the search process depends on the appropriate choice of algorithms, heuristics, and strategies for the specific problem at hand.

Search Algorithm Terminologies:

- o **Search:** Searching is a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:
 - a. **Search Space:** Search space represents a set of possible solutions, which a system may have.
 - a. **Start State:** It is a state from where agent begins **the search**.
 - b. **Goal test:** It is a function which observe the current state and returns whether the goal state is achieved or not.

Search tree: A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.

Actions: It gives the description of all the available actions to the agent.

Transition model: A description of what each action do, can be represented as a transition model.

Path Cost: It is a function which assigns a numeric cost to each path.

Solution: It is an action sequence which leads from the start node to the goal node.

Optimal Solution: If a solution has the lowest cost among all solutions.

Properties of Search Algorithms:

Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

Completeness: A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.

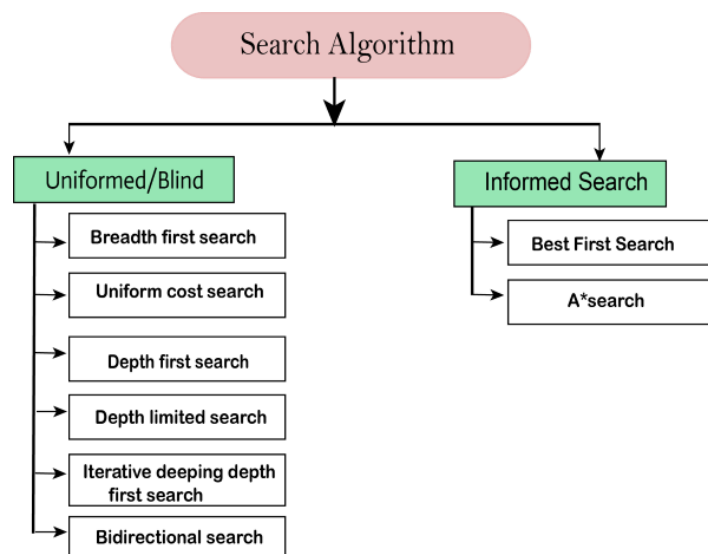
Optimality: If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.

Time Complexity: Time complexity is a measure of time for an algorithm to complete its task.

Space Complexity: It is the maximum storage space required at any point during the search, as the complexity of the problem.

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Uninformed/Blind Search:

The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search. It examines each node of the tree until it achieves the goal node.

It can be divided into five main types:

- o Breadth-first search
- o Uniform cost search
- o Depth-first search
- o Iterative deepening depth-first search
- o Bidirectional Search

Informed Search

Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.

A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

Informed search can solve much complex problem which could not be solved in another way.

An example of informed search algorithms is a traveling salesman problem.

1. Greedy Search
2. A* Search

Uninformed Search Algorithms

Uninformed search is a class of general-purpose search algorithms which operates in brute force-way. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.

Following are the various types of uninformed search algorithms:

1. Breadth-first Search
2. Depth-first Search
3. Depth-limited Search
4. Iterative deepening depth-first search
5. Uniform cost search
6. Bidirectional Search

1. Breadth-first Search:

- o Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- o BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- o The breadth-first search algorithm is an example of a general-graph search algorithm.
- o Breadth-first search implemented using FIFO queue data structure.

Advantages:

- o BFS will provide a solution if any solution exists.

- o If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

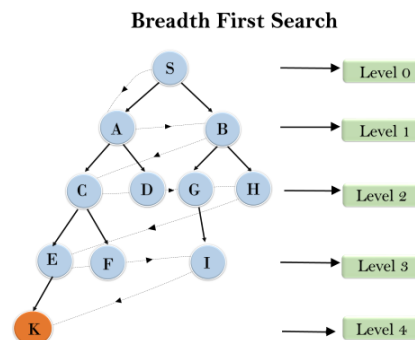
Disadvantages:

- o It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- o BFS needs lots of time if the solution is far away from the root node.

Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S---> A--->B--->C--->D--->G--->H--->E--->F--->I--->K



Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d = depth of shallowest solution and b is a node at every state.

$$T(b) = 1 + b^1 + b^2 + \dots + b^d = O(b^d)$$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. Depth-first Search

- o Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- o It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.

- o DFS uses a stack data structure for its implementation.
- o The process of the DFS algorithm is similar to the BFS algorithm.

Note: *Backtracking is an algorithm technique for finding all possible solutions using recursion.*

Advantage:

- o DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- o It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

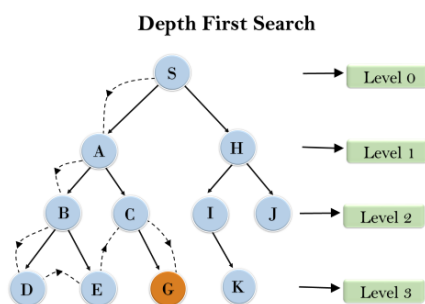
- o There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- o DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Example:

In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

Root node--->Left node ----> right node.

It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$T(n) = 1 + n^2 + n^3 + \dots + n^m = O(n^m)$$

Where, m = maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is $O(bm)$.

Optimal: DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

3. Depth-Limited Search Algorithm:

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- o Standard failure value: It indicates that problem does not have any solution.
- o Cutoff failure value: It defines no solution for the problem within a given depth limit.

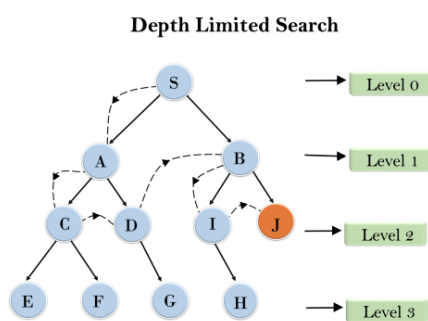
Advantages:

Depth-limited search is Memory efficient.

Disadvantages:

- o Depth-limited search also has a disadvantage of incompleteness.
- o It may not be optimal if the problem has more than one solution.

Example:



Completeness: DLS search algorithm is complete if the solution is above the depth-limit.

Time Complexity: Time complexity of DLS algorithm is $O(b^l)$.

Space Complexity: Space complexity of DLS algorithm is $O(b \times l)$.

Optimal: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $l > d$.

4. Uniform-cost Search Algorithm:

Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge. The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost. Uniform-cost search expands nodes according to their path costs from the root node. It can be used to solve any graph/tree where the optimal cost is in demand. A uniform-cost search algorithm is implemented by the priority queue. It gives maximum priority to the lowest cumulative cost. Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

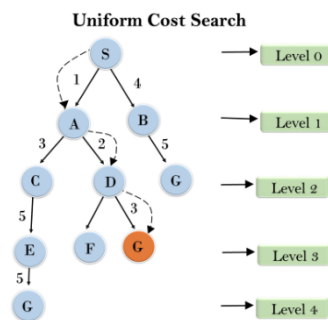
Advantages:

- o Uniform cost search is optimal because at every state the path with the least cost is chosen.

Disadvantages:

- o It does not care about the number of steps involve in searching and only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.

Example:



Completeness:

Uniform-cost search is complete, such as if there is a solution, UCS will find it.

Time Complexity:

Let C^* is **Cost of the optimal solution**, and ϵ is each step to get closer to the goal node. Then the number of steps is $= C^*/\epsilon + 1$. Here we have taken $+1$, as we start from state 0 and end to C^*/ϵ .

Hence, the worst-case time complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

Space Complexity:

The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

Optimal:

Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

5. Iterative deepening depth-first Search:

The iterative deepening algorithm is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.

The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

Advantages:

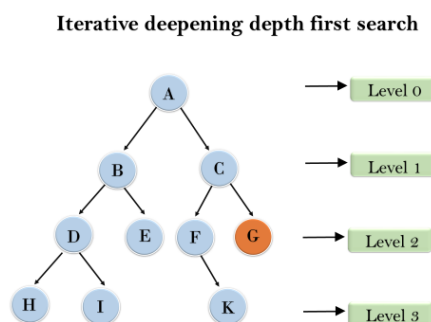
- o It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.

Disadvantages:

- o The main drawback of IDDFS is that it repeats all the work of the previous phase.

Example:

Following tree structure is showing the iterative deepening depth-first search. IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:



1'st Iteration-----> A

2'nd Iteration-----> A, B, C

3'rd Iteration----->A, B, D, E, C, F, G

4'th Iteration----->A, B, D, H, I, E, C, F, K, G

In the fourth iteration, the algorithm will find the goal node.

Completeness:

This algorithm is complete is if the branching factor is finite.

Time Complexity:

Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.

Space Complexity:

The space complexity of IDDFS will be $O(bd)$.

Optimal:

IDDFS algorithm is optimal if path cost is a non- decreasing function of the depth of the node.

Informed Search Algorithms

So far we have talked about the uninformed search algorithms which looked through search space for all possible solutions of the problem without having any additional knowledge about search space. But informed search algorithm contains an array of knowledge such as how far we are from the goal, path cost, how to reach to goal node, etc. This knowledge help agents to explore less to the search space and find more efficiently the goal node.

The informed search algorithm is more useful for large search space. Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Heuristics function: Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal. The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

Admissibility of the heuristic function is given as:

$$1. \quad h(n) \leq h^*(n)$$

Here $h(n)$ is heuristic cost, and $h^*(n)$ is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value $h(n)$. It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.

On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues until a goal state is found.

In the informed search we will discuss two main algorithms which are given below:

- o **Best First Search Algorithm(Greedy search)**
- o **A* Search Algorithm**

1.) Best-first Search Algorithm (Greedy Search):

Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

$$1. \quad f(n) = g(n).$$

Where, $h(n)$ = estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm:

- o **Step 1:** Place the starting node into the OPEN list.
- o **Step 2:** If the OPEN list is empty, Stop and return failure.

- o **Step 3:** Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.
- o **Step 4:** Expand the node n , and generate the successors of node n .
- o **Step 5:** Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- o **Step 6:** For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- o **Step 7:** Return to Step 2.

Advantages:

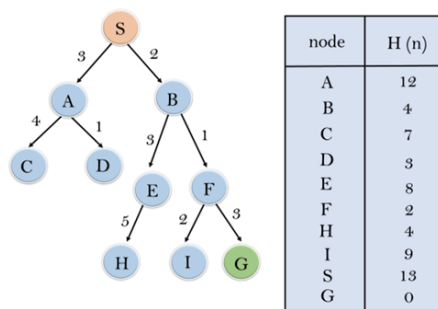
- o Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- o This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages:

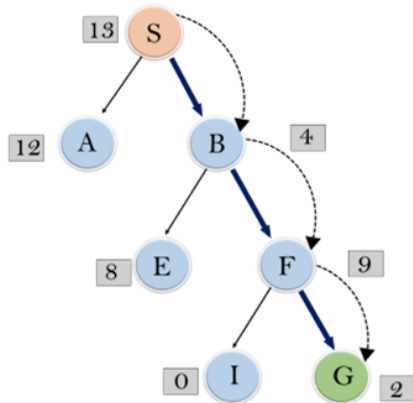
- o It can behave as an unguided depth-first search in the worst case scenario.
- o It can get stuck in a loop as DFS.
- o This algorithm is not optimal.

Example:

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function $f(n)=h(n)$, which is given in the below table.



In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: **S----> B----->F-----> G**

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

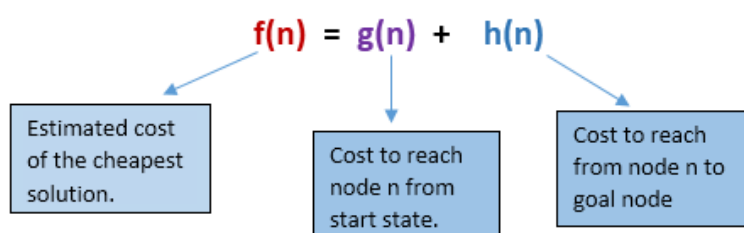
Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

2.) A* Search Algorithm:

A* search is the most commonly known form of best-first search. It uses heuristic function $h(n)$, and cost to reach the node n from the start state $g(n)$. It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses $g(n)+h(n)$ instead of $g(n)$.

In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number**.



At each point in the search space, only those node is expanded which have the lowest value of $f(n)$, and the algorithm terminates when the goal node is found.

Algorithm of A* search:

Step 1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function ($g+h$), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n' , check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest $g(n')$ value.

Step 6: Return to **Step 2**.

Advantages:

- o A* search algorithm is the best algorithm than other search algorithms.
- o A* search algorithm is optimal and complete.
- o This algorithm can solve very complex problems.

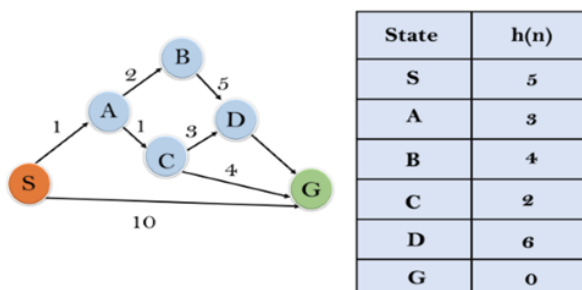
Disadvantages:

- o It does not always produce the shortest path as it mostly based on heuristics and approximation.
- o A* search algorithm has some complexity issues.
- o The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

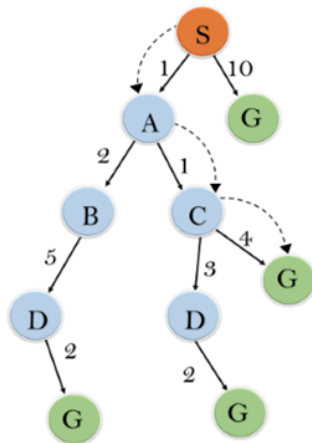
Example:

In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the $f(n)$ of each state using the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



Solution:



Initialization: $\{(S, 5)\}$

Iteration1: $\{(S \rightarrow A, 4), (S \rightarrow G, 10)\}$

Iteration2: $\{(S \rightarrow A \rightarrow C, 4), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration3: $\{(S \rightarrow A \rightarrow C \rightarrow G, 6), (S \rightarrow A \rightarrow C \rightarrow D, 11), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration 4 will give the final result, as $S \rightarrow A \rightarrow C \rightarrow G$ it provides the optimal path with cost 6.

Points to remember:

- o A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- o The efficiency of A* algorithm depends on the quality of heuristic.
- o A* algorithm expands all nodes which satisfy the condition $f(n) \leq l_i$

Complete: A* algorithm is complete as long as:

- o Branching factor is finite.
- o Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

- o **Admissible:** the first condition requires for optimality is that $h(n)$ should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- o **Consistency:** Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.

Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d . So the time complexity is $O(b^d)$, where b is the branching factor.

Space Complexity: The space complexity of A* search algorithm is $O(b^d)$

Generate-and-test

Generate and Test Search is a heuristic search technique based on Depth First Search with Backtracking which guarantees to find a solution if done systematically and there exists a solution. In this technique, all the solutions are generated and tested for the best solution. It ensures that the best solution is checked against all possible generated solutions.

It is also known as British Museum Search Algorithm as it's like looking for an exhibit at random or finding an object in the British Museum by wandering randomly.

The evaluation is carried out by the heuristic function as all the solutions are generated systematically in generate and test algorithm but if there are some paths which are most unlikely to lead us to result then they are not considered. The heuristic does this by ranking all the alternatives and is often effective in doing so. Systematic Generate and Test may prove to be ineffective while solving complex problems. But there is a technique to improve in complex cases as well by combining generate and test search with other techniques so as to reduce the search space. For example in Artificial Intelligence Program DENDRAL we make use of two techniques, the first one is Constraint Satisfaction Techniques followed by Generate and Test Procedure to work on reduced search space i.e. yield an effective result by working on a lesser number of lists generated in the very first step.

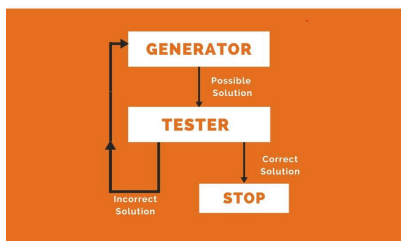
Algorithm

Generate a possible solution. For example, generating a particular point in the problem space or generating a path for a start state.

Test to see if this is a actual solution by comparing the chosen point or the endpoint of the chosen path to the set of acceptable goal states

If a solution is found, quit. Otherwise go to Step 1

DIAGRAMMATIC REPRESENTATION



Properties of Good Generators:

The good generators need to have the following properties:

Complete: Good Generators need to be complete i.e. they should generate all the possible solutions and cover all the possible states. In this way, we can guaranty our algorithm to converge to the correct solution at some point in time.

Non Redundant: Good Generators should not yield a duplicate solution at any point of time as it reduces the efficiency of algorithm thereby increasing the time of search and making the time complexity exponential. In fact, it is often said that if solutions appear several times in the depth-first search then it is better to modify the procedure to traverse a graph rather than a tree.

Informed: Good Generators have the knowledge about the search space which they maintain in the form of an array of knowledge. This can be used to search how far the agent is from the goal, calculate the path cost and even find a way to reach the goal.

Let us take a simple example to understand the importance of a good generator. Consider a pin made up of three 2 digit numbers i.e. the numbers are of the form,



In this case, one way to find the required pin is to generate all the solutions in a brute force manner for example,



The total number of solutions in this case is $(100)^3$ which is approximately 1M. So if we do not make use of any informed search technique then it results in exponential time complexity. Now let's say if we generate 5 solutions every minute. Then the total numbers generated in 1 hour are $5 \times 60 = 300$ and the total number of solutions to be generated are 1M. Let us consider the brute force search technique for example linear search whose average time complexity is $N/2$. Then on an average, the total number of the solutions to be generated are approximately 5 lakhs. Using this technique even if you work for about 24 hrs a day then also you will need 10 weeks to complete the task.

Now consider using heuristic function where we have domain knowledge that every number is a prime number between 0-99 then the possible number of solutions are $(25)^3$ which is approximately 15,000. Now consider the same case that you are generating 5 solutions every minute and working for 24 hrs then you can find the solution in less than 2 days which was being done in 10 weeks in the case of uninformed search.

We can conclude for here that if we can find a good heuristic then time complexity can be reduced gradually. But in the worst-case time and space complexity will be exponential. It all depends on the generator i.e. better the generator lesser is the time complexity.

Interesting Real Life Examples:

If a sufficient number of monkeys were placed in front of a set of typewriters, and left alone long enough, then they would eventually produce all the works of Shakespeare.

Dendral which infers the structure of organic compounds using NMR spectrogram also uses plan-generate-test.

Don't miss your chance to ride the wave of the data revolution! Every industry is scaling new heights by tapping into the power of data. Sharpen your skills and become a part of the hottest trend in the 21st century.

